Hams Aljohani
Lab 4 big data

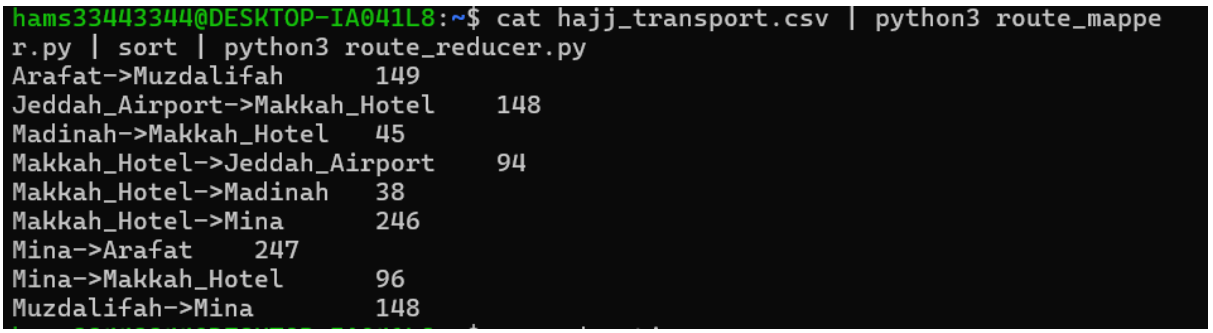
Exercise 2

A:
```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()

    if not line or line.startswith("trip_id"):
        continue

    parts = line.split(",")

    try:
        origin = parts[2]
        destination = parts[3]
        passengers = int(parts[5])

        route = origin + "->" + destination

        print(f"{route}\t{passengers}")

    except:
        continue
```

B:
```
#!/usr/bin/env python3
import sys

current_route = None
total = 0

for line in sys.stdin:
    line = line.strip()

    try:
        route, value = line.split("\t")
        value = int(value)
    except:
        continue

    if route == current_route:
        total += value
```

```
    else:
        if current_route:
            print(f"{current_route}\t{total}")

        current_route = route
        total = value

if current_route:
    print(f"{current_route}\t{total}")
```

C:

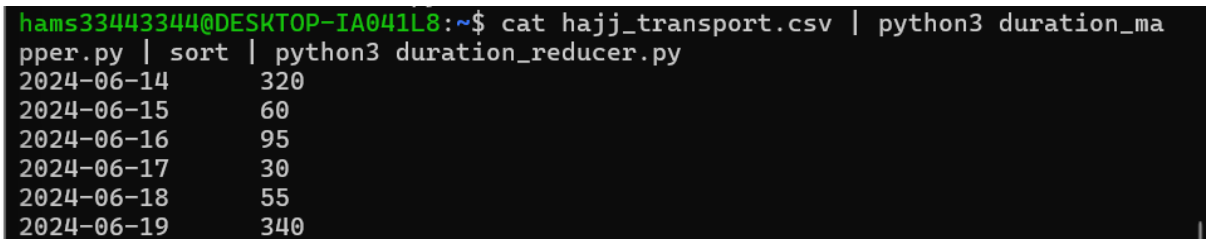


D:
```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()

    if not line or line.startswith("trip_id"):
        continue

    parts = line.split(",")

    try:
        date = parts[1]
        duration = int(parts[6])

        print(f"{date}\t{duration}")

    except:
        continue
```

E:
```
#!/usr/bin/env python3
import sys

current_date = None
```

```
max_duration = 0

for line in sys.stdin:
    line = line.strip()

    try:
        date, duration = line.split("\t")
        duration = int(duration)
    except:
        continue

    if date == current_date:
        max_duration = max(max_duration, duration)

    else:
        if current_date:
            print(f"{current_date}\t{max_duration}")

        current_date = date
        max_duration = duration

if current_date:
    print(f"{current_date}\t{max_duration}")
```

F:



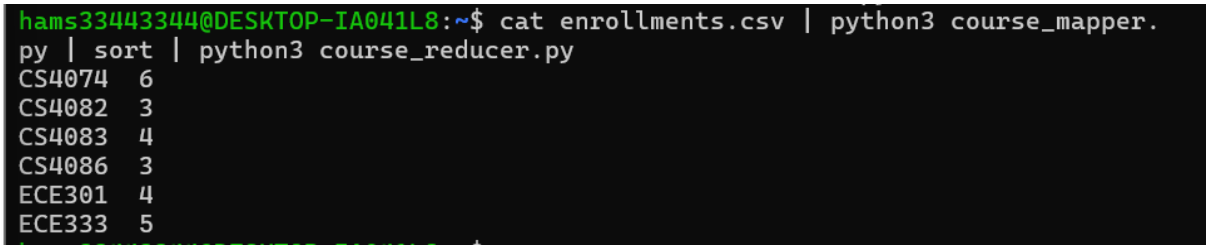
Exercise 3

A:
```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line=line.strip()

    if not line or line.startswith("student_id"):
        continue

    parts=line.split(",")
```

```
    try:
        course=parts[3]
        print(f"{course}\t1")
    except:
        continue
```

B:
```
#!/usr/bin/env python3
import sys

current=None
count=0

for line in sys.stdin:

    course,value=line.strip().split("\t")
    value=int(value)

    if course==current:
        count+=value

    else:
        if current:
            print(f"{current}\t{count}")

        current=course
        count=value

if current:
    print(f"{current}\t{count}")
```

C:



D:
```
#!/usr/bin/env python3
import sys

gpa_map={
"A+":4.0,"A":4.0,"B+":3.5,"B":3.0,"C+":2.5,"C":2.0
```

```
}

for line in sys.stdin:

    line=line.strip()

    if not line or line.startswith("student_id"):
        continue

    parts=line.split(",")

    try:
        dept=parts[2]
        grade=parts[6]

        gpa=gpa_map.get(grade,0)

        print(f"{dept}\t{gpa}")
    except:
        continue

E: #!/usr/bin/env python3
import sys

current=None
total=0
count=0

for line in sys.stdin:

    dept,value=line.strip().split("\t")
    value=float(value)

    if dept==current:
        total+=value
        count+=1

    else:
        if current:
            print(f"{current}\t{total/count:.2f}")

        current=dept
        total=value
        count=1
```

```
if current:
    print(f"{current}\t{total/count:.2f}")
```

F:

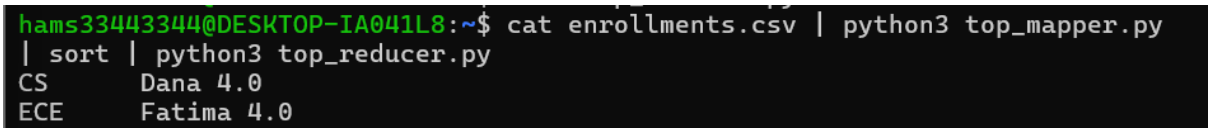
```
hams33443344@DESKTOP-IA041L8:~$ cat enrollments.csv | python3 gpa_mapper.py
| sort | python3 gpa_reducer.py
CS      3.79
ECE     3.27
```

G:
```
#!/usr/bin/env python3
import sys

gpa_map={"A+":4.0,"A":4.0,"B+":3.5,"B":3.0,"C+":2.5,"C":2.0}

for line in sys.stdin:

    line=line.strip()

    if not line or line.startswith("student_id"):
        continue

    parts=line.split(",")

    try:
        name=parts[1]
        dept=parts[2]
        grade=parts[6]

        gpa=gpa_map.get(grade,0)

        print(f"{dept}\t{name}:{gpa}")
    except:
        continue
```

```
#!/usr/bin/env python3
import sys

current=None
best_gpa=0
best_student=""
```

```
for line in sys.stdin:

    dept,data=line.strip().split("\t")
    name,gpa=data.split(":")
    gpa=float(gpa)

    if dept==current:

        if gpa>best_gpa:
            best_gpa=gpa
            best_student=name

    else:

        if current:
            print(f"{current}\t{best_student} {best_gpa}")

        current=dept
        best_gpa=gpa
        best_student=name

if current:
    print(f"{current}\t{best_student} {best_gpa}")
```

H:



Exercise 4

Dataset file
https://www.kaggle.com/datasets/gregorut/videogamesales?utm_source=chatgpt.com

Screenshot: head -5 and wc -l of your dataset

```
hams33443344@DESKTOP-IA041L8:~$ head -5 vgsales.csv
Rank,Name,Platform,Year,Genre,Publisher,NA_Sales,EU_Sales,JP_Sales,Other_Sal
es,Global_Sales
1,Wii Sports,Wii,2006,Sports,Nintendo,41.49,29.02,3.77,8.46,82.74
2,Super Mario Bros.,NES,1985,Platform,Nintendo,29.08,3.58,6.81,0.77,40.24
3,Mario Kart Wii,Wii,2008,Racing,Nintendo,15.85,12.88,3.79,3.31,35.82
4,Wii Sports Resort,Wii,2009,Sports,Nintendo,15.75,11.01,3.28,2.96,33
hams33443344@DESKTOP-IA041L8:~$ wc -l vgsales.csv
16599 vgsales.csv
```

.

```
hams33443344@DESKTOP-IA041L8:~$ hdfs dfs -ls /user/student/lab3/ex4_input
Found 1 items
-rw-r--r--   1 hams33443344 supergroup    1355781 2026-03-12 10:08 /user/stu
dent/lab3/ex4_input/vgsales.csv
```

Analysis 1: mapper.py + reducer.py code:

Mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()

    if line.startswith("Rank"):
        continue

    parts = line.split(",")

    try:
        genre = parts[4]
        sales = float(parts[10])
        print(f"{genre}\t{sales}")
    except:
        continue
```

Reducer.py

```
#!/usr/bin/env python3
import sys

current_genre = None
total_sales = 0

for line in sys.stdin:
```

```
    line = line.strip()
    genre, value = line.split("\t")
    value = float(value)

    if genre == current_genre:
        total_sales += value
    else:
        if current_genre:
            print(current_genre, total_sales)
        current_genre = genre
        total_sales = value

if current_genre:
    print(current_genre, total_sales)
```

Analysis 1: local test output screenshot:

```
hams33443344@DESKTOP-IA041L8:~$ cat vgsales.csv | python3 genre_mapper.py |
sort | python3 genre_reducer.py
O-Kata" 0.0
Wii & PC Versions)" 0.76
1994 0.0
1998 0.01
2000 0.0
2001 0.14
2002 0.06
2003 0.08
2004 0.01
2005 0.16999999999999998
2006 0.0
2007 0.09
2008 0.01
2009 0.07
2010 0.05
2011 0.29000000000000004
2013 0.0
2014 0.0
2015 0.0
2016 0.0
3DS 0.0
Action 1744.25
Adventure 234.5499999999997
DS 0.03
Fighting 448.9000000000001
GBA 0.0
GC 0.0
Misc 806.0100000000002
N/A 0.02
PC 0.0
PS 0.0
PS2 0.0
PS3 0.0
PSP 0.0
Platform 829.4800000000001
Puzzle 242.83
Racing 731.67
Role-Playing 926.2900000000001
Shooter 1032.0500000000004
```

```
Simulation 390.1099999999999
Sports 1330.530000000001
Strategy 174.00000000000003
Wii 0.0
X360 0.0
XB 0.0
hams33443344@DESKTOP-IA041L8:~$
```

Analysis 1: Hadoop output (first 15 lines)

```
hams33443344@DESKTOP-IA041L8:~$ hdfs dfs -cat /user/student/lab3/output1/par
t-00000 | head
O-Kata" 0.0
Wii & PC Versions)" 0.76
1994 0.0
1998 0.01
2000 0.0
2001 0.14
2002 0.060000000000000005
2003 0.08
2004 0.01
2005 0.16999999999999996
```

Analysis 2: mapper.py + reducer.py code

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()

    if line.startswith("Rank"):
        continue

    parts = line.split(",")

    try:
        platform = parts[2]
        print(f"{platform}\t1")
    except:
        continue
```

reducer.py

```
#!/usr/bin/env python3
import sys

current = None
count = 0

for line in sys.stdin:
    line = line.strip()
    platform, value = line.split("\t")
    value = int(value)
```

```
    if platform == current:
        count += value
    else:
        if current:
            print(current, count)
        current = platform
        count = value

if current:
    print(current, count)
```

Analysis 2: local test output screenshot

```
hams33443344@DESKTOP-IA041L8:~$ cat vgsales.csv | python3 platform_mapper.py
 | sort | python3 platform_reducer.py
A-Kata 1
Badman! What Did I Do to Deserve This?" 1
Be Glorious!" 1
Be Gracious!" 1
Boku to Himitsu no Chizu" 1
Camera 4
Diego 2
Dood!" 1
Edd n Eddy: Jawbreakers!" 1
Edd n Eddy: The Mis-Edventures" 4
Fashion and Friends" 1
Futatabi" 2
Ginkaku no Inbou" 1
Hajime Mashita" 1
Hidden Dragon" 3
Himawari no Shoujo" 1
Inc. Scream Arena" 1
Inc. Scream Team" 1
Inc." 2
Inc.: Mega MicroGame$" 1
Inc.: Mega Party Game$" 1
Kai-lan: New Year's Celebration" 1
Kai-lan: Super Game Day" 2
Keiyaku Shikkou Dechai masuu" 1
Kimi ni" 2
Koi Utsutsu: Setsugetsuka Koi Emaki" 1
Koiseyo Otome! Love is Power!!!" 1
Kono Sekai ni Kami-sama ga Iru to suru Naraba." 1
Kore ga Otoko Dearu!" 1
London 1969" 1
Mahjong & Tangram" 1
Mon Meilleur Ami" 1
Monkey Do!" 1
My Friend 1
My Lord!? 2" 1
My Love" 2
My Way (US sales)" 1
Myths & Legends" 1
```

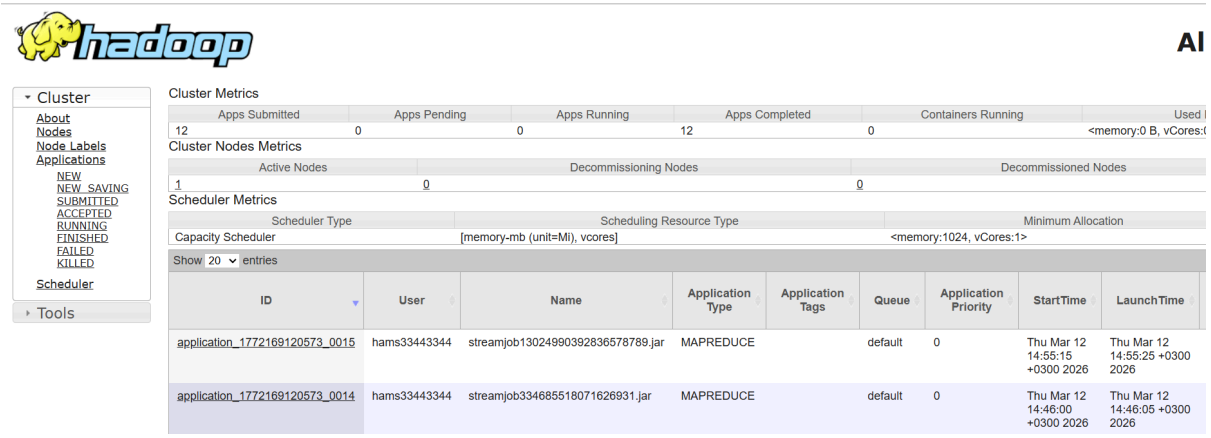
```
Nine Persons 1
PS2 5
PS3 9
Pikachu!" 1
Sakini Ike" 1
Sea 1
Secret Agent 1
Set 2
Set & Style!" 2
She Wrote" 1
Shinitamou Koto Nakare" 1
Shinitamou koto Nakare" 1
Shutsujin! Koisen" 1
Sun 1
Ten e Kaere" 1
The Bad and The Bugly" 1
The Doug Williams Edition" 1
The Witch and The Wardrobe" 5
Trade 1
Ware ga Ichizoku Twin Pack" 1
Ware ga Ichizoku" 1
Ware ga Ichizoku: Tasogare Polarstar" 1
Yoga & Pilates Workout" 1
Zenigata ni wa Koi o" 1
but wrong system)" 2
000: Dawn of War II - Chaos Rising" 1
000: Dawn of War II - Retribution" 1
000: Dawn of War II" 1
000: Dawn of War" 1
000: Dawn of War: Soulstorm" 1
000: Fire Warrior" 1
000: Space Marine" 3
000: Squad Command" 2
2600 133
3DO 3
3DS 507
DC 50
DS 2142
GB 98
GBA 815
GC 551
```

```
GEN 26
GG 1
N64 318
NES 98
NG 12
Otome" 1
PC 951
PCFX 1
PS 1192
PS2 2144
PS3 1324
PS4 336
PSP 1200
PSV 411
SAT 172
SCD 6
SNES 239
TG16 2
WS 6
Wii 1317
WiiU 143
X360 1258
XB 820
XOne 213
big Adventures: Ham-Ham Challenge" 1
```

Analysis 2: Hadoop output (first 15 lines)

```
hams33443344@DESKTOP-IA041L8:~$ hdfs dfs -cat /user/student/lab3/output2/par
t-00000 | head
A-Kata 1
Badman! What Did I Do to Deserve This?" 1
Be Glorious!" 1
Be Gracious!" 1
Boku to Himitsu no Chizu" 1
Camera 4
Diego 2
Dood!" 1
Edd n Eddy: Jawbreakers!" 1
Edd n Eddy: The Mis-Edventures" 4
```

Screenshot: YARN Web UI showing completed jobs



Report

1. Dataset Description

The dataset used in this exercise is the **Video Game Sales Dataset** obtained from Kaggle. Source URL:
https://www.kaggle.com/datasets/gregorut/videogamesales

The dataset contains **16,599 rows** and **11 columns** representing video game sales information across different platforms and regions. Each row corresponds to a specific video game and its associated sales statistics.

The dataset columns include:

- **Rank** – the ranking of the game based on global sales
- **Name** – the title of the video game
- **Platform** – the gaming platform (e.g., PS2, Wii, Xbox)
- **Year** – the year the game was released
- **Genre** – the genre category of the game (Action, Sports, etc.)
- **Publisher** – the company that published the game
- **NA_Sales** – sales in North America (in millions)
- **EU_Sales** – sales in Europe (in millions)
- **JP_Sales** – sales in Japan (in millions)
- **Other_Sales** – sales in other regions (in millions)
- **Global_Sales** – total worldwide sales (in millions)

Some data quality issues were observed. Certain rows contain missing values such as **"N/A" in the Year column**, and some game titles include commas which can affect simple CSV parsing. These issues were handled by skipping problematic rows in the mapper.

---

2. Analysis Design

Analysis 1: Total Global Sales per Genre

The goal of the first analysis was to determine **which video game genres generate the highest global sales**.

The **key** chosen was the **Genre** column because we want to group all games belonging to the same genre together.

The **value** used was **Global_Sales**, which represents the total worldwide sales of each game.

The aggregation method used in the reducer was **summation**, where all sales values belonging to the same genre were added together to compute the total sales per genre.

Analysis 2: Number of Games per Platform

The second analysis aimed to determine **how many games exist for each gaming platform**.

The **key** selected was the **Platform** column because the analysis groups games by their platform.

The **value** used was **1**, representing one game record.

The aggregation method used in the reducer was **counting**, where the reducer sums all the values to determine the total number of games available on each platform.

---

3. Implementation

The MapReduce programs were implemented using **Python with Hadoop Streaming**.

The mapper for the first analysis extracts the **Genre** and **Global_Sales** fields and outputs them as key-value pairs. The reducer then sums the sales values for each genre.

The mapper for the second analysis extracts the **Platform** field and outputs a value of **1** for each game. The reducer counts the occurrences of each platform.

The mapper includes basic data cleaning operations such as:

• skipping the header row
• handling missing values using try/except
• ignoring invalid rows that cannot be parsed

Local testing was performed using the following command:

```
cat vgsales.csv | python3 genre_mapper.py | sort | python3
genre_reducer.py
```

The Hadoop streaming execution command used was:

```
hadoop jar
$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming*.jar \
-input /user/student/lab3/ex4_input/vgsales.csv \
-output /user/student/lab3/output1 \
-mapper genre_mapper.py \
-reducer genre_reducer.py
```

A similar command was used for the second analysis with different mapper and reducer scripts.

---

4. Results & Discussion

The output of the first MapReduce program shows the **total global sales per genre**. Example results include:

```
Action 1744.25
Sports 1330.53
Shooter 1032.05
Role-Playing 926.29
Platform 829.48
```

These results indicate that **Action and Sports games generate the highest global sales** among all genres.

The output of the second MapReduce program shows the **number of games released per platform**. Example results include:

```
DS 2142
PS2 2144
PS3 1324
Wii 1317
X360 1258
```

This shows that platforms such as **Nintendo DS and PlayStation 2 have the largest number of games in the dataset**.

To verify the correctness of the results, two rows from the dataset were manually checked. For example, games labeled with the genre **Sports** were traced in the dataset and their global sales values were summed to confirm the reducer's output. Similarly, several records with platform **PS2** were counted manually to confirm the platform count results.

---

5. Challenges & Lessons Learned

One of the main challenges encountered during this lab was dealing with **CSV formatting issues**, especially game titles containing commas. This caused incorrect column parsing when using simple string splitting.

Another challenge was understanding how Hadoop Streaming processes input data and how the mapper and reducer communicate through standard input and output.

These issues were solved by implementing **error handling in the mapper** and skipping problematic rows using try/except blocks.

Additionally, learning how to debug MapReduce programs using **local testing before running Hadoop jobs**proved extremely useful.

If this project were repeated in the future, a better approach would be to use a more robust CSV parser or preprocess the dataset before running MapReduce to ensure cleaner input data.